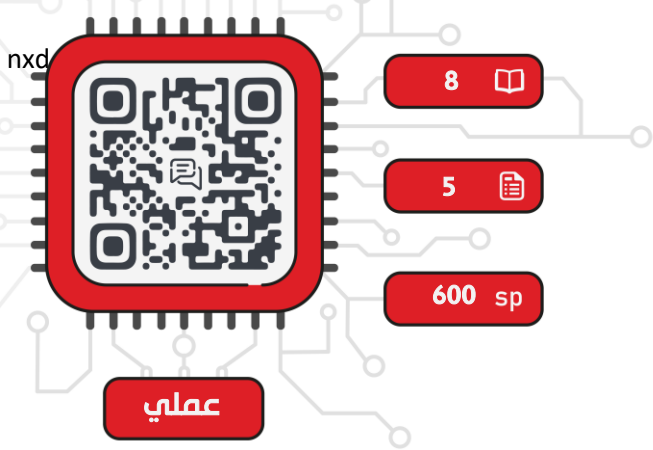




بسم الله الرحمن الرحيم

تمرين 1:

لدينا مصفوفة A تحتوي على اعداد صحيحة و تنتهي بالعدد 0.
اكتب برنامج بلغة MIPS يمر على عناصر المصفوفة حتى يصل إلى العنصر 0، و في كل مرة يقارن العنصر الحالي مع العدد 4، اذا كان العنصر أصغر او يساوي يستبدل قيمته ب 0، و الا يضاعف قيمته
(اكتب بجانب كل تعليمة زائفة التعليمات الحقيقة المكافئة لها)

حل اربسييزي :

.data

A : .word 1, 2, 3, 4, 5, 6, 7, 0

.text

la \$s1, A # lui \$at, upper(A) ori \$s1, \$at, lower(A)

li \$s2, 4 # ori \$s2, \$0, 4

loop:

lw \$s3, 0(\$s1) # \$s3 <- M[\$s1]
beq \$s3, \$0, End # if (\$s3 == 0) {go to "End"} else {continue}
ble \$s3, \$s2, less # slt \$1, \$s2, \$s3 beq \$1, \$0, less

if \$s3 > \$s4
add \$s4, \$s3, \$s3 # \$s4 = \$s3 * 2
sw \$s4, 0(\$s1) # \$s4 -> M[\$s1]
j after # go to after

if \$s3 <= \$s4

less:

sw \$0, 0(\$s1) # 0 -> M[\$s1]

after:

addi \$s1,\$s1, 4 # \$s1 = \$s1 + 4 (moving to the next element in A)

j loop # repeat the loop

End:

محتوى الذاكرة قبل تنفيذ البرنامج :

Data Segment							
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)
0x10010000	1	2	3	4	5	6	7
0x10010020	0	0	0	0	0	0	0

محتوى الذاكرة بعد تنفيذ البرنامج:

Data Segment							
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)
0x10010000	0	0	0	0	10	12	14
0x10010020	0	0	0	0	0	0	0

جميع القيم التي أصغر أو تساوي 4 تم استبدالها بـ 0 وجميع القيم التي أكبر من 4 تمت مضاعفتها.

■ تمرين 2:

لدينا مصفوفتان A و B لهما نفس عدد العناصر حيث تحتوي كل منها على اعداد صحيحة و تنتهي بالعدد 0، و لدينا اعداد.

اكتب برنامج بلغة MIPS يمر على عناصر المصفوفتان، و في كل مرة يقارن العنصر الحالي من المصفوفة A مع العنصر الحالي من المصفوفة B، اذا كان العنصر من A اكبر من العنصر من B نزيد العدد بمقدار 1، و الا نطرح من العدد 1

(اكتب بجانب كل تعليمة قائمة التعليمات الحقيقية المكافئة لها)

حل أربسيزي:

.data

A : .word 2, 3, 4, 0

B : .word 1, 5, 3, 0

.text

la \$s1, A # lui \$at, upper(A) ori \$s1, \$at, lower(A)

la \$s2, B # lui \$at, upper(B) ori \$s1, \$at, lower(B)

addi \$t0, \$0, 0 # the counter

loop:

lw \$s3, 0(\$s1) # \$s3 <- M[\$s1]

lw \$s4, 0(\$s2) # \$s4 <- M[\$s2]

beq \$s3, \$0, End # if (\$s3 == 0){go to End (end the code)} else {continue}

bgt \$s3, \$s4, greater # slt \$1, \$s4, \$s3 bne \$1, \$0, greater

 # if (\$s3 > \$s4){go to greater} else {continue}

 # if \$s3 <= \$s4

addi \$t0, \$t0, -1 # \$t0 = \$t0 - 1

j after # go to after

 # if \$s3 > \$s4

greater:

addi \$t0, \$t0, 1 # \$t0 = \$t0 + 1

after:

addi \$s1, \$s1, 4 # \$s1 = \$s1 + 4 (moving to the next element in A)

addi \$s2, \$s2, 4 # \$s2 = \$s2 + 4 (moving to the next element in B)

j loop # repeat the loop

End:

قيمة العداد بعد تنفيذ الكود : \$t0 = 1



"Burning like embers,
falling tender
Longing for the days of no
surrender...."

■ تمرين 3 :

لدينا مصفوفتان M و C، حيث ان المصفوفة M تحتوي على 5 اعداد صحيحة و تنتهي بالعدد 0، و المصفوفة C محجوزة ولا تحتوي اي قيم
اكتب برنامج بلغة MIPS يمر على عناصر المصفوفة M من البداية حتى الوصول الي الرقم صفر، و في كل خطوة عند العنصر $M[i]$ يقوم بالتالي:

- 1- يحسب مجموع العنصر الحالي و العنصر الذي يليه ($M[i] + M[i+1]$)
- 2- يضاعف قيمة العنصر الحالي ($M[i]*2$)
- 3- يقارن بين مجموع العنصرين و العدد المضاعف
- 4- يخزن القيمة الاكبر في المصفوفة C عند نفس الموضع i

(اكتب بجانب كل تعليمة زائفة التعليمات الحقيقة المكافئة لها)

.data

A : .word 1, 2, 3, 1, 0

B : .word 0:5

.text

la \$s1, A # lui \$at, upper(A) ori \$s1, \$at, lower(A)

la \$s2, B # lui \$at, upper(B) ori \$s1, \$at, lower(B)

loop:

lw \$s3, 0(\$s1) # \$s3 <- M[\$s1]

beq \$s3, \$0, End # if (\$s3 == 0) {go to End (end the code)} else {continue}

lw \$s4, 4(\$s1) # \$s4 <- M[4 + \$s1]

add \$t1, \$s3, \$s4 # \$t1 = \$s3 + \$s4 (\$t1 = M[i] + M[i+1])

add \$t2, \$s3, \$s3 # \$t2 = \$s3 + \$s3 (\$t2 = M[i] * 2)

bge \$t1, \$t2, greater # slt \$1, \$t1, \$t2 beq \$1, \$0, greater

if (M[i] + M[i+1] >= M[i] * 2) {go to "greater"} else {continue}

M[i] + M[i+1] < M[i] * 2 (\$t1 < \$t2)

sw \$t2, 0(\$s2) # \$t2 -> M[\$s2] (C[i] = M[i] * 2)

j after # go to after

M[i] + M[i+1] >= M[i] * 2 (\$t1 >= \$t2)

greater:

sw \$t1, 0(\$s2) # \$t1 -> M[\$s2] (C[i] = M[i] + M[i+1])

after:

addi \$s1,\$s1, 4 # \$s1 = \$s1 + 4 (moving to the next element in M)

addi \$s2,\$s2, 4 # \$s2 = \$s2 + 4 (moving to the next element in C)

j loop # repeat the loop

End:

محتوى الذاكرة قبل تنفيذ البرنامج :

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	1	2	3	1	0	0	0	0
0x10010020	0	0	0	0	0	0	0	0

محتوى الذاكرة بعد تنفيذ البرنامج:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	1	2	3	1	0	3	5	6
0x10010020	2	0	0	0	0	0	0	0



"Feeling like the very last drops of an ink pen"

~ The End ~